

TypeScript - Interfaces e Types

Guia conciso de estudo cobrindo conceitos essenciais, sintaxes, exemplos práticos e comparações.

ÍNDICE DO CONTEÚDO

1. Interfaces básicas
2. Propriedades opcionais (?)
3. Propriedades somente leitura (readonly)
4. Interfaces com arrays de objetos
5. Funções tipadas com interface
6. Herança de interfaces (extends)
7. Type Alias
8. Interseção de types (&)
9. Interfaces vs Types: diferenças

1. Interfaces básicas

[readonly.ts](#) / [interfaces.ts](#)

Uma `interface` descreve o contrato que um objeto deve seguir: quais propriedades existem e seus respectivos tipos.

```
interface ILojaProps {
  nome: string;
  endereco: string;
  status: boolean;
}

const BurguerK: ILojaProps = {
  nome: "Burguer K",
  endereco: "Rua logo ali",
  status: true,
};
```

Interfaces também tipam parâmetros de funções, permitindo desestruturação direta:

```
function novaLoja({ nome, endereco, status }: ILojaProps): void {
  console.log(`Loja ${nome} criada com sucesso!`);
}

novaLoja({ nome: "Subway", endereco: "Rua dez", status: true });
```

2. Propriedades opcionais (?)

[opcional.ts](#)

O operador `?` torna o preenchimento de uma propriedade não obrigatório no objeto.

```
interface CadastroProps {
  nome?: string;
  email: string;
  status: boolean;
}

const novoUsuario: CadastroProps = {
  email: "matheus@teste.com",
  status: true,
}; // "nome" é opcional
```

3. Propriedades somente leitura (readonly)

readonly.ts

O modificador `readonly` impede a reatribuição de valor a uma propriedade após sua inicialização.

```
interface ProdutoProps {
  readonly id: string;
  nome: string;
  descricao: string;
}

let produto1: ProdutoProps = {
  id: "1",
  nome: "Tenis Nike",
  descricao: "Super tenis descolado",
};

produto1.nome = "Tenis Nike 2.0"; // Permitido
// produto1.id = "2" // Erro: "id" é readonly
```

4. Interfaces com arrays de objetos

interfaces_types.ts

Interfaces podem aninhar tipos complexos, como coleções de outras interfaces.

```
interface TecnologiaProps {
  id: string;
  nome: string;
  descricao?: string;
}

interface NomesProps {
  tecnologia: TecnologiaProps[];
}

let frontend: NomesProps = {
  tecnologia: [
    { id: "12", nome: "ReactJS", descricao: "Biblioteca UI" },
    { id: "43", nome: "VueJs", descricao: "Framework Frontend" },
  ],
};
```

5. Funções tipadas com interface

funcao.ts

Interfaces definem propriedades funcionais ou assinaturas diretas de chamada (call signatures):

```
interface CursoProps {
  id: string;
  nome: string;
  preco: number;
  promocao: (preco: number) => void;
}

// Call Signature direta para função
interface SomaProps {
  (valor1: number, valor2: number): number;
}

let somaNumeros: SomaProps = (valor1, valor2) => valor1 + valor2;
const resultado = somaNumeros(15, 10);
```

6. Herança de interfaces (extends)

extends.ts

```
interface JogoProps {
  readonly id: string;
  nome: string;
  descricao: string;
}

interface DLC extends JogoProps {
  novoConteudo: string[];
}
```

7. Type Alias & 8. Interseção (&)

type_alias.ts / type_intersection.ts

type cria apelidos para primitivos, uniões ou objetos. A interseção (&) combina tipos.

```
type Uuid = number | string | null;

type Info = { id: number; nome: string; };
type Categoria = { slug: string; };

type ProdutoInfo = Info & Categoria; // Interseção
```

9. Interfaces vs Types: Comparações

interfaces_types.ts

Recurso	Interface	Type Alias
Descrever Objetos	Sim	Sim
Herança / Composição	extends	& (Interseção)
Tipos Primitivos / Uniões	Não possui suporte	Sim (ex: string number)
Declaration Merging	Sim (múltiplas declarações se somam)	Não (gera erro de duplicidade)

Resumo Prático de Escolha

- **Use Interface:** para modelar contratos de objetos, classes ou APIs com possibilidade de expansão por *declaration merging*.
- **Use Type:** para representar uniões, primitivos nomeados, tuplas e composições com o operador & .